

## MODULE 9

# GitHub Copilot for Spring Boot Development

AI-assisted development from requirement to tested microservice.

Copilot  
fundamentals

Backend  
prompting

Spring Boot  
code  
generation

Controllers and  
services

AI-assisted  
testing

Refactoring and  
quality

Enterprise  
guardrails

Hands-on  
microservice

# LEARNING OBJECTIVES

*Prompting, code generation, tests, refactoring and enterprise best practices.*

## BY THE END OF THIS MODULE, YOU CAN

1. Explain where GitHub Copilot fits
2. Write effective backend prompts
3. Provide useful project context
4. Generate controllers and services
5. Generate DTOs, repositories and validation
6. Create tests with AI assistance
7. Identify missing test scenarios
8. Refactor existing code
9. Use Copilot for troubleshooting
10. Apply enterprise AI guardrails
11. Build a microservice iteratively

**THE GOAL** Not who generates the most code — who builds correct, secure software faster.

**KEY TAKEAWAY** Use AI to produce correct, maintainable, secure software more efficiently.

## FOUR SECTIONS

### 1 AI-assisted fundamentals

Copilot

Developer

Project context

### 2 AI-assisted implementation

DTO

Controller

Service

Repository

Validation

### 3 AI-assisted quality

Tests

Code review

Refactoring

Troubleshooting

### 4 Enterprise governance

Security

Human  
review

CI/CD

Policies

Production

# KEY TERMS FOR THIS MODULE

*Plain definitions for the words we'll use most.*

## 1 **GitHub Copilot**

An AI coding assistant in the editor that suggests code, tests and explanations.

## 2 **Prompt**

The instruction and context you give the AI to get a result.

## 3 **Context**

The files, code and rules the AI can see when it answers.

## 4 **Custom instructions**

A repository file of team conventions that Copilot applies to every chat.

## 5 **Hallucination**

Confident AI output that is wrong, such as a method or library that doesn't exist.

## 6 **Human in the loop**

A person reviews and approves AI output before it's used.

## 7 **Refactoring**

Improving the structure of code without changing what it does.

## 8 **Guardrails**

Rules and controls that limit what AI tools may see, generate or change.

**KEY TAKEAWAY** Agree on the words first — every later slide builds on them.

# FULL FORMS USED IN THIS MODULE

*Every abbreviation on these slides, spelled out.*

|              |                                                 |             |                             |             |                                 |
|--------------|-------------------------------------------------|-------------|-----------------------------|-------------|---------------------------------|
| <b>AI</b>    | Artificial Intelligence                         | <b>DTO</b>  | Data Transfer Object        | <b>REST</b> | Representational State Transfer |
| <b>API</b>   | Application Programming Interface               | <b>HTTP</b> | HyperText Transfer Protocol | <b>SQL</b>  | Structured Query Language       |
| <b>CI</b>    | Continuous Integration                          | <b>JPA</b>  | Jakarta Persistence API     | <b>URI</b>  | Uniform Resource Identifier     |
| <b>CI/CD</b> | Continuous Integration /<br>Continuous Delivery | <b>JSON</b> | JavaScript Object Notation  | <b>YAML</b> | YAML Ain't Markup Language      |
| <b>DB</b>    | Database                                        | <b>PR</b>   | Pull Request                |             |                                 |

**KEY TAKEAWAY** When an abbreviation isn't clear, the full form is on this slide.

# FROM MODULE 8 TO MODULE 9

*We know how to design, build, ship and run services — now build them faster with AI.*

## MODULES 1–8 GAVE US

- ✓ Service design and REST contracts
- ✓ Layered Spring Boot services
- ✓ Data, messaging and resilience patterns
- ✓ Pipelines with tests and scans
- ✓ Containers running on OpenShift

**NOW** Use GitHub Copilot to go faster — without lowering the bar.

## OUR PATH THROUGH MODULE 9

- |   |                                         |        |
|---|-----------------------------------------|--------|
| 1 | What Copilot is and how it fits         | Part 1 |
| 2 | Prompts that give real context          | Part 1 |
| 3 | Generate controllers, services and DTOs | Part 2 |
| 4 | Persistence and error handling          | Part 2 |
| 5 | Tests, refactoring and review           | Part 3 |
| 6 | Troubleshooting with Copilot            | Part 3 |
| 7 | Security, governance and responsibility | Part 4 |

**KEY TAKEAWAY** Everything from Modules 1–8 still applies — Copilot helps you apply it faster.

# PART 1: AI-ASSISTED FUNDAMENTALS

*Before generating code, understand the assistant and how to talk to it.*



## SO FAR

- ✓ We know what good Spring Boot code looks like
- ✓ Pipelines test and scan every change
- ✓ Architecture decisions are already made

## NEXT QUESTION

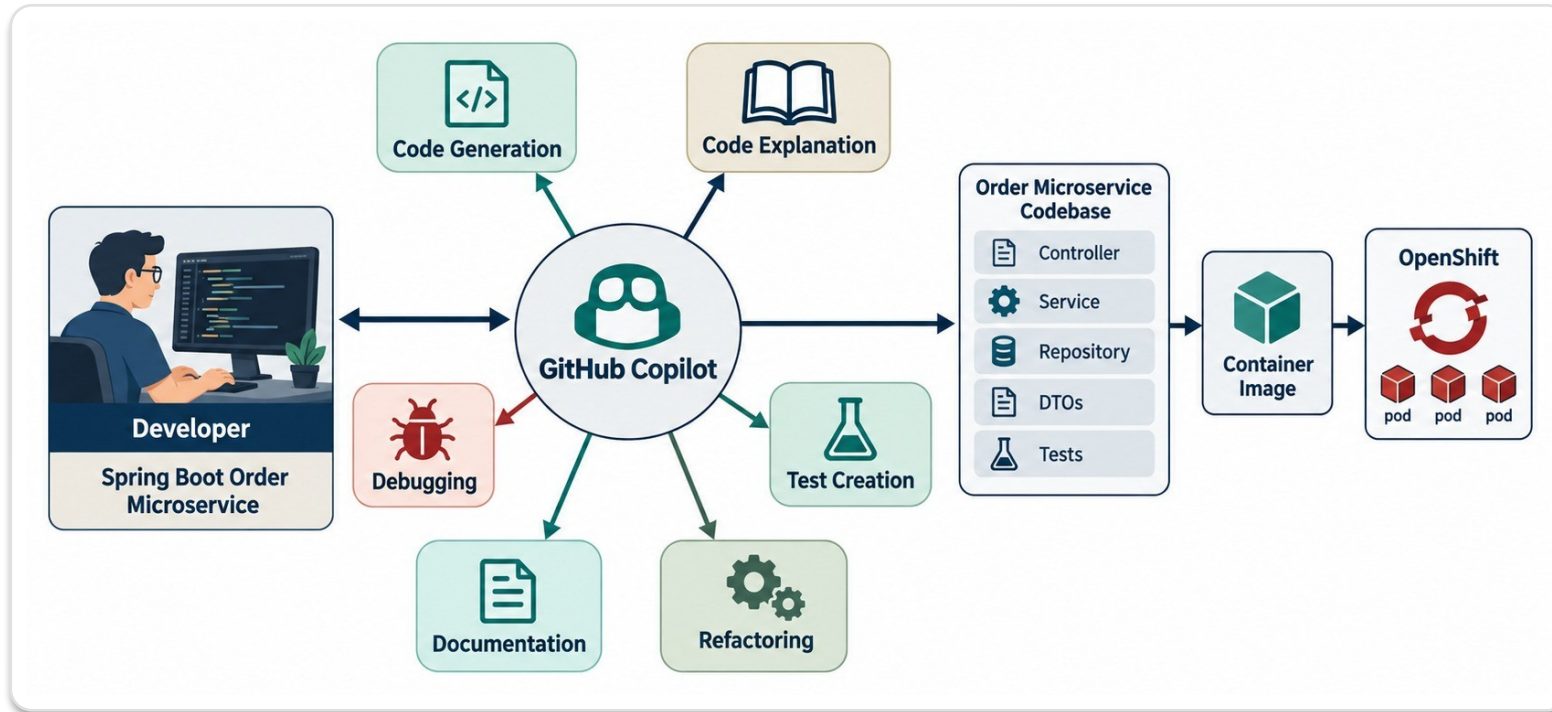
**What can Copilot actually do for a backend developer — and how do we ask it well?**

## IN THIS PART

- 1 What Copilot is — and isn't
- 2 The AI-assisted workflow
- 3 Copilot across a Spring Boot project
- 4 Prompting fundamentals and structure
- 5 Repository instructions — in code
- 6 Exercise: improve a weak prompt

# WHAT IS GITHUB COPILOT?

*AI coding assistant capabilities and where it fits in the development workflow.*



## WAYS TO USE IT

Inline suggestions

Chat

Agent mode

Code review

Explain code

Generate tests

## WHAT IT IS NOT

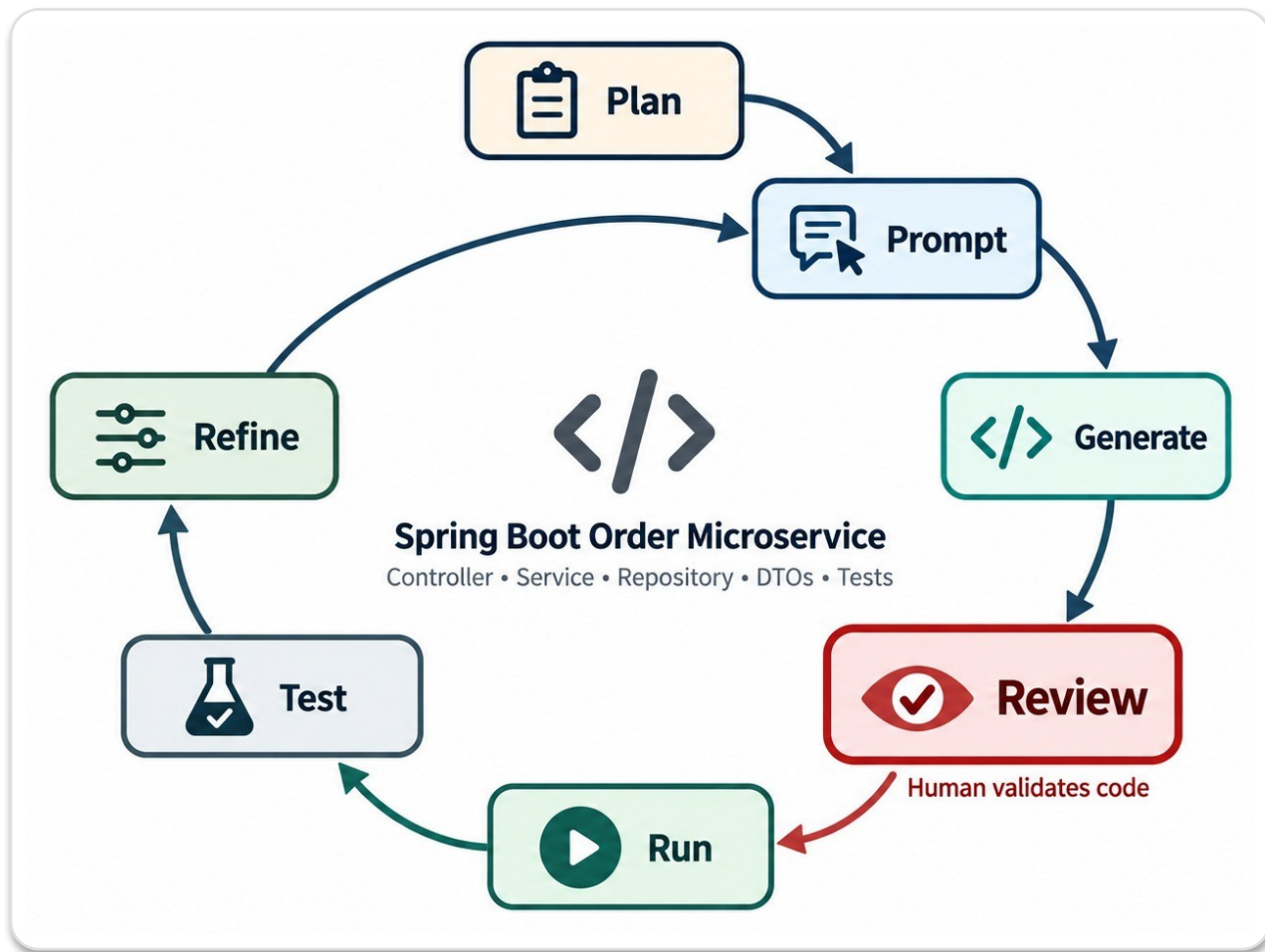
- ✗ Not an architect
- ✗ Not a security reviewer
- ✗ Not accountable for defects

**RULE** Every suggestion is a candidate to review.

**KEY TAKEAWAY** Copilot assists the developer — the developer stays responsible for correctness.

# HOW COPILOT SUPPORTS THE DEVELOPER WORKFLOW

*Plan → prompt → generate → review → run → test → refine.*



## Plan

Know the requirement and the design before prompting.

## Review and run

Read every line; compile and run before trusting it.

## Refine

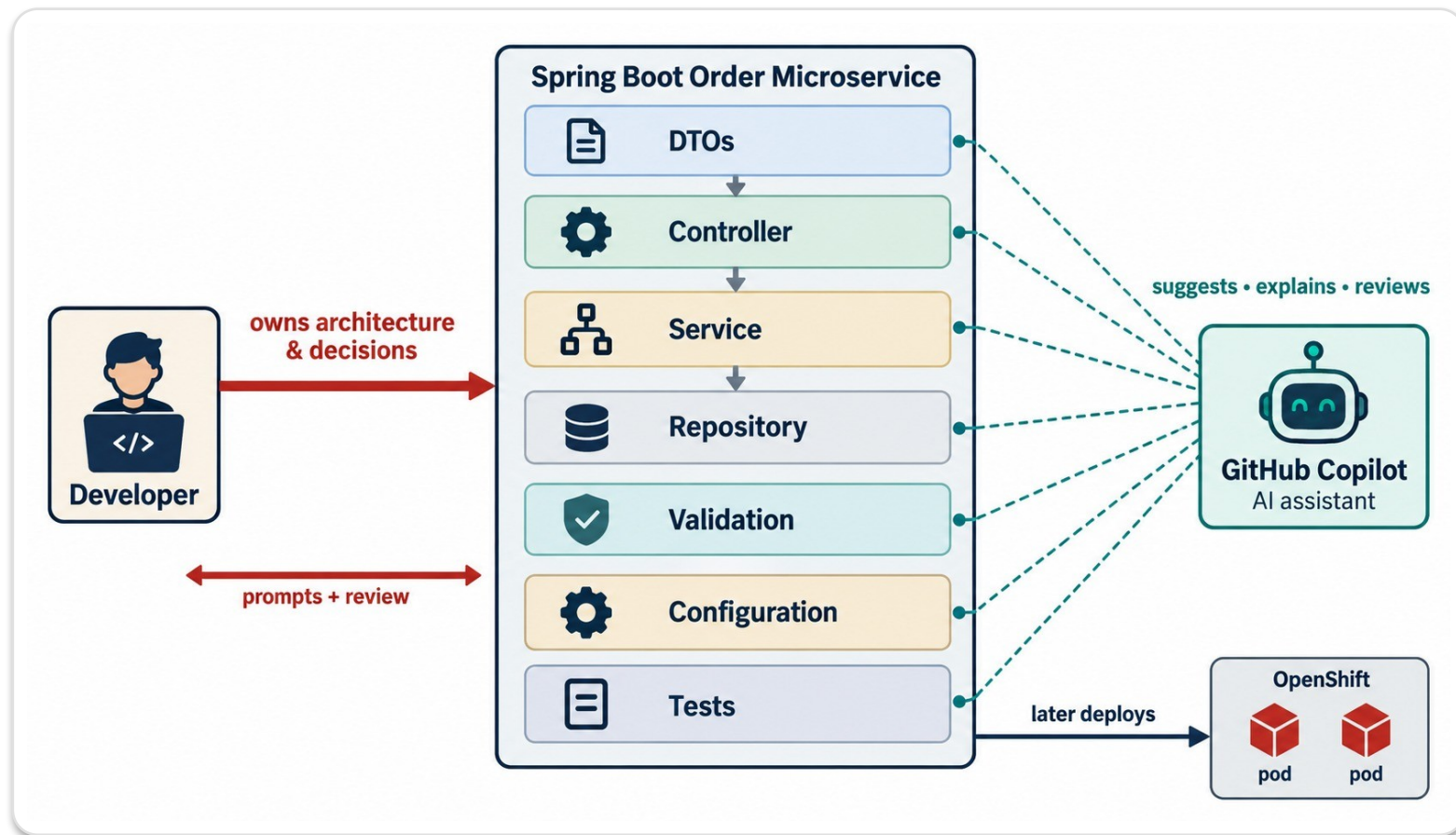
Feed errors and failing tests back into the next prompt.

**NOT ONE-SHOT** Small loops beat one giant prompt.

**KEY TAKEAWAY** AI-assisted development is iterative: plan, prompt, generate, review, run, test, refine.

# COPILOT IN A SPRING BOOT PROJECT

*Help across controllers, services, repositories, DTOs, tests and config.*



## Great at

DTOs, mappings, boilerplate, test scaffolds, YAML and docs.

## Review closely

Business rules, transactions, security and queries.

**ASSIST ≠ DESIGN** Boundaries and contracts stay your decision.

**KEY TAKEAWAY** Copilot helps implement each layer of an architecture you already understand.

# PROMPTING FUNDAMENTALS FOR BACKEND DEVELOPMENT

*Context, intent, constraints and output format.*



## Context

Spring Boot 3, Java 21, the package and existing classes.

## Task

Exactly what to build — one layer or method at a time.

## Constraints

Validation, status codes, no entities in responses.

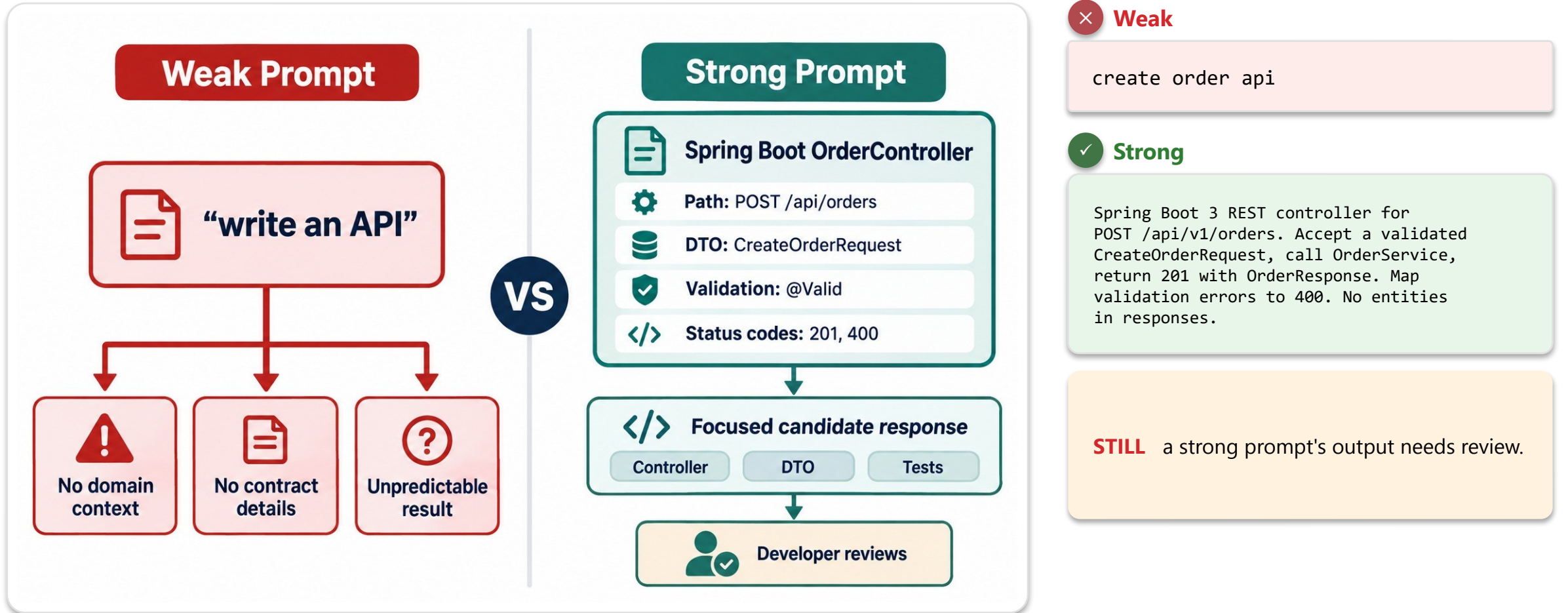
## Output

Which files, and whether to include tests.

**KEY TAKEAWAY** Prompt quality matters: give context, task, constraints and the output you expect.

# WEAK PROMPT VS. STRONG PROMPT

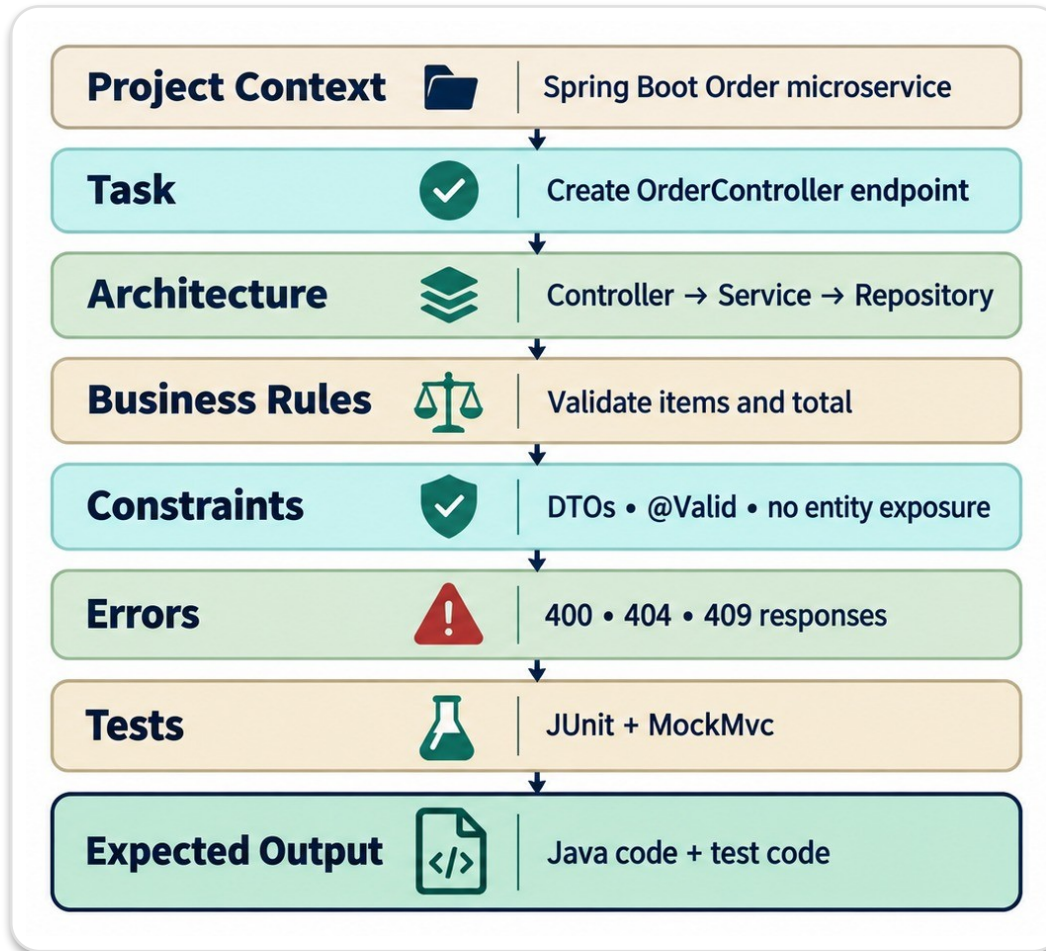
*Compare vague and precise prompts.*



**KEY TAKEAWAY** Better prompts reduce ambiguity, but generated output still needs review.

# PROMPT STRUCTURE FOR SPRING BOOT TASKS

*Context → task → constraints → standards → expected result.*



## A REUSABLE TEMPLATE

- ✓ Project context
- ✓ Task
- ✓ Architecture and layer
- ✓ Business rules
- ✓ Errors and status codes
- ✓ Tests expected

**WHY** Everyone on the team gets consistent results.

**KEY TAKEAWAY** A reusable prompt structure beats generic prompt-engineering tricks.

# GIVE COPILOT YOUR STANDARDS ONCE

*Repository instructions apply to every chat; a structured prompt handles the task.*

## [.github/copilot-instructions.md](#)

```
# Order Service conventions
- Java 21, Spring Boot 3, Maven.
- Layers: controller -> service -> repository. No logic in controllers.
- Request/response DTOs are Java records; never return JPA entities.
- Validate with Jakarta Validation; errors via GlobalExceptionHandler.
- Constructor injection only. No field @Autowired.
- Tests: JUnit 5, Mockito, AssertJ; @WebMvcTest for controllers.
- Never log request bodies, tokens or account numbers.
```

### Instructions file

Team standards apply to every Copilot chat in the repo.

## ✓ A structured task prompt

```
Context: OrderService in com.example.order.service (open file).
Task: add cancelOrder(Long orderId).
Rules: only CREATED or PAID orders can be cancelled;
      PAID orders publish OrderCancelled for a refund.
Errors: missing order -> OrderNotFoundException (404);
      other status -> InvalidOrderStateException (409).
Constraints: @Transactional; follow existing naming.
Output: the method plus unit tests for all three cases.
```

### Prompt

Context, task, rules, errors, constraints, output.

### Result

Code that fits the codebase — still reviewed.

**KEY TAKEAWAY** Put team conventions in the repository once, and give each task a structured prompt.

# EXERCISE: IMPROVE A WEAK PROMPT

*Rewrite a vague request into a prompt a teammate could reuse.*

## PRACTICE EXERCISE

10 MIN

### SCENARIO

A developer typed this into Copilot Chat and got an insecure, entity-returning controller:

```
make a transfer endpoint
```

### YOUR TASKS

- 1 List what Copilot had to guess
- 2 Rewrite the prompt with context, task, rules, errors and output
- 3 Name two things you'd still check in the result

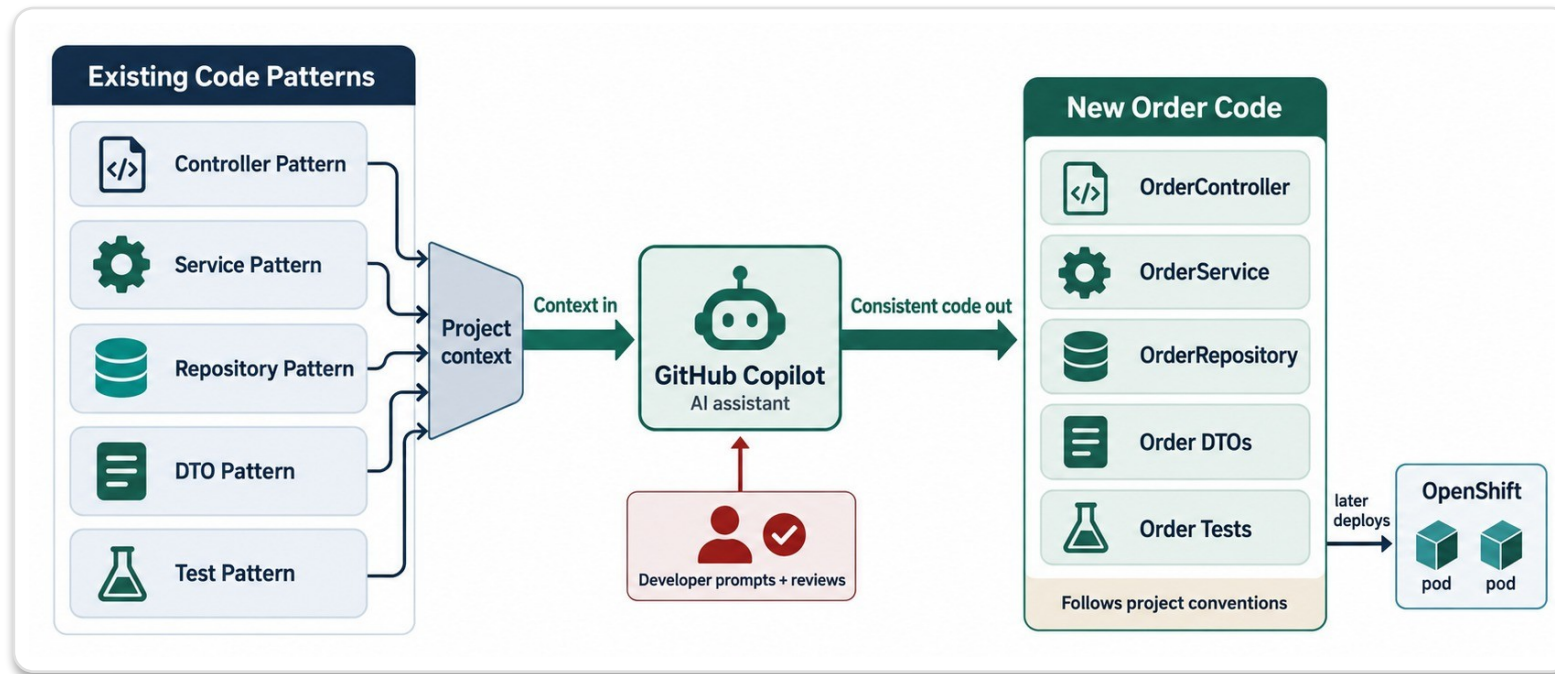
### EXPECTED OUTCOME

- ✓ Guessed: path, DTOs, validation, status codes, security
- ✓ POST /api/v1/transfers with a validated TransferRequest
- ✓ Idempotency-Key; 201, 400, 404, 409 defined
- ✓ No entities, no logged account numbers
- ✓ Still check: authorization and the business rules

**KEY TAKEAWAY** A good prompt states context, rules, errors and output — and the result still gets reviewed.

# USING EXISTING CODE AS CONTEXT

*Extend existing classes without breaking conventions.*



## Open the right files

Keep a similar controller, service and test open while prompting.

## Say it

"Follow the pattern in PaymentController and its tests."

**GOAL** New code that looks like the team wrote it.

**KEY TAKEAWAY** Give Copilot the existing patterns so new code fits the codebase you already have.

# PART 2: AI-ASSISTED IMPLEMENTATION

*Generate one Spring Boot layer at a time — and review each one.*



## SO FAR

- ✓ Copilot drafts; the developer decides
- ✓ Structured prompts carry the rules
- ✓ Instructions files hold team standards

## NEXT QUESTION

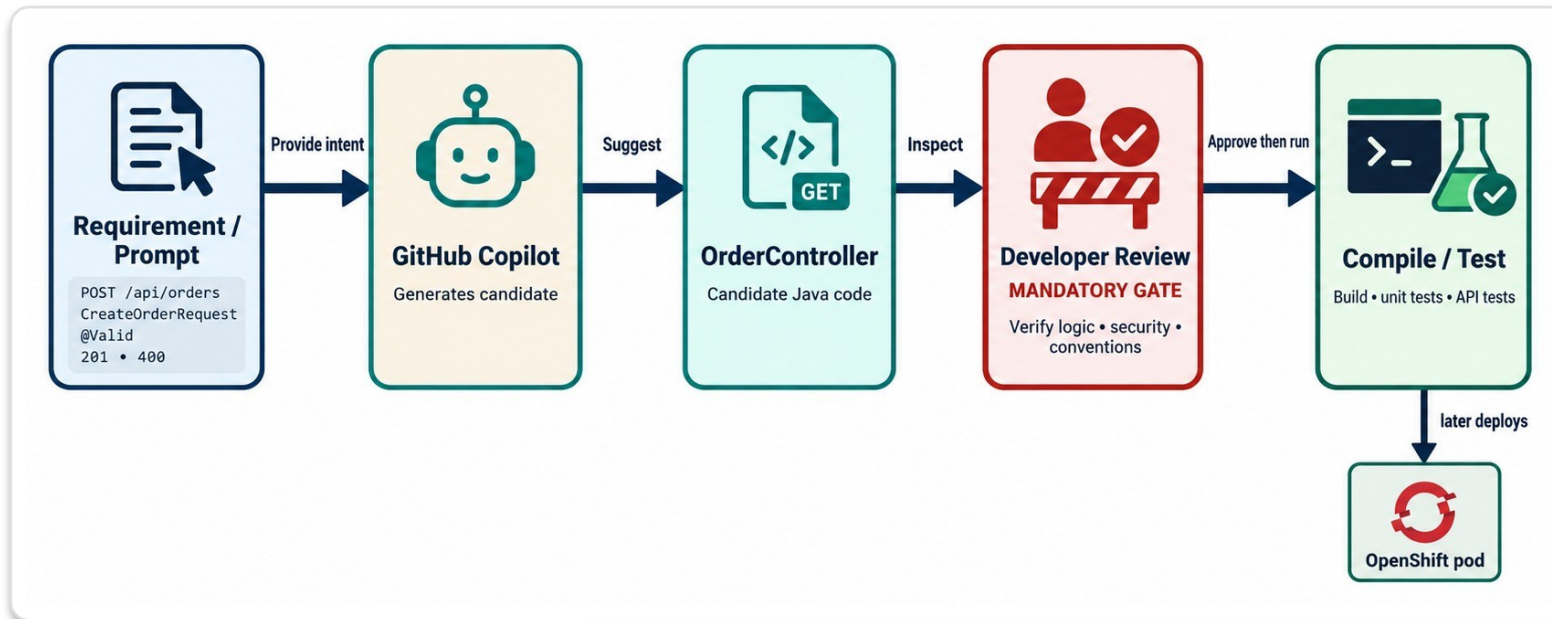
**How do we use Copilot to build real service layers without importing bugs?**

## IN THIS PART

- 1 Generating a REST controller — with a review
- 2 Service-layer logic — with a review
- 3 Exercise: review AI-generated transfer code
- 4 DTOs and API contracts
- 5 Persistence code
- 6 Validation and error handling

# GENERATING A SPRING BOOT REST CONTROLLER

*HTTP request → OrderController → OrderService → response DTO → HTTP response.*



## REVIEW CHECKLIST

- ✓ Resource paths and methods
- ✓ `@Valid` on request bodies
- ✓ 201 with Location on create
- ✓ No business logic
- ✓ No entities returned

**ONE LAYER** at a time — then compile and test.

**KEY TAKEAWAY** Generate one layer at a time — then review architecture and API before you compile and test.

# REVIEWING A GENERATED CONTROLLER

*Copilot's first draft compiled — review turned it into a proper API.*

## ✗ First draft

```
@RestController
public class OrderController {
    @Autowired OrderRepository repo;

    @PostMapping("/createOrder")
    public Order create(@RequestBody Order order) {
        if (order.getTotal().signum() <= 0)
            return null;
        return repo.save(order);
    }
}
```

### Found

Verb path, entity in and out, repository in the controller.

### Found

Field injection, logic in the controller, null instead of 400.

## ✓ After review

```
@RestController
@RequestMapping("/api/v1/orders")
class OrderController {
    private final OrderService orders;
    OrderController(OrderService orders) { this.orders = orders; }

    @PostMapping
    ResponseEntity<OrderResponse> create(
        @Valid @RequestBody CreateOrderRequest req) {
        OrderResponse created = orders.create(req);
        return ResponseEntity
            .created(URI.create("/api/v1/orders/" + created.id()))
            .body(created);
    }
}
```

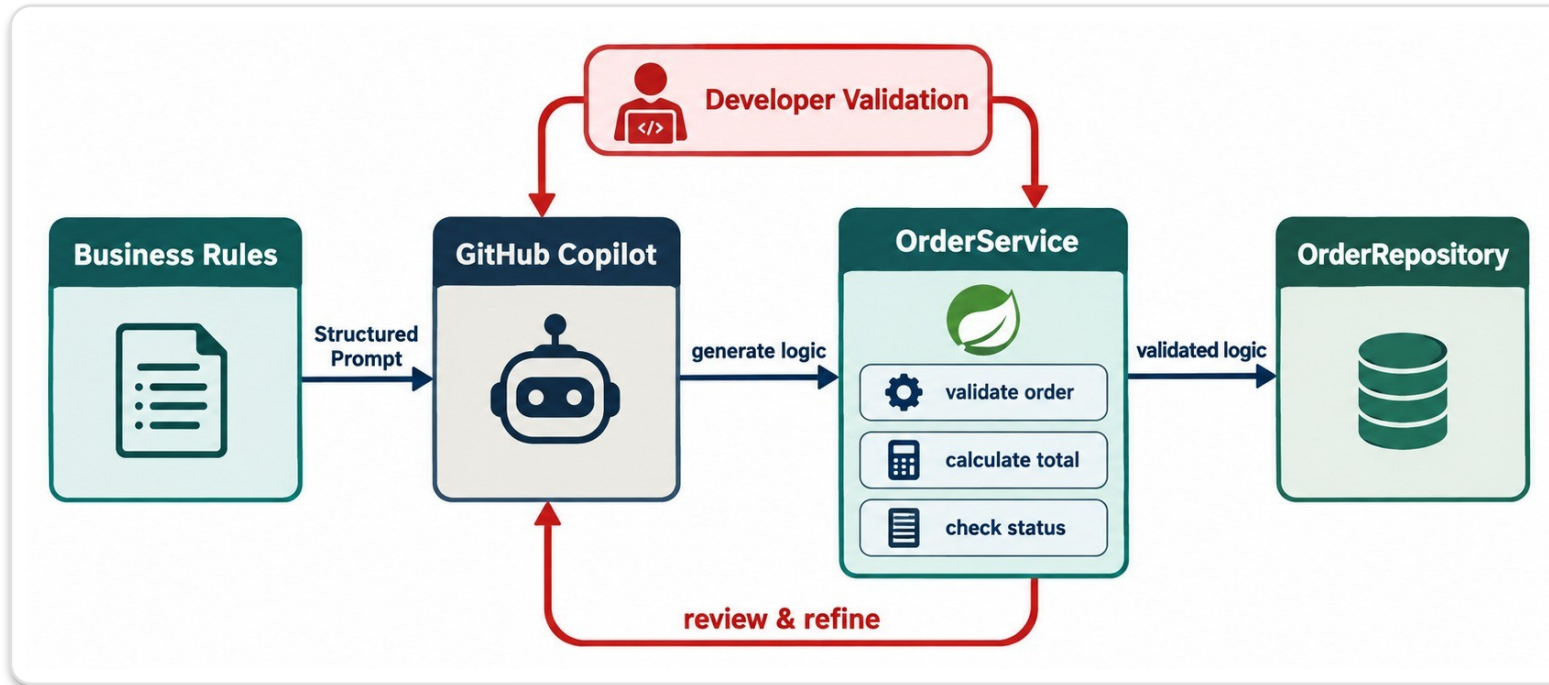
### Fixed

Resource path, DTOs, @Valid, service call, 201 Created.

**KEY TAKEAWAY** Generated code that compiles can still break every rule from Modules 2 and 3 — review it.

# GENERATING SERVICE-LAYER LOGIC

*Business rules, validation, dependency injection and separation of concerns.*



## VERIFY BY HAND

- ! Every business rule present
- ! Correct transaction boundary
- ! Idempotency for money movement
- ! Right exceptions

### Why riskiest

Missing rules compile fine — they only fail in production.

**LESSON** Incomplete requirements give incomplete code.

**KEY TAKEAWAY** Incomplete requirements produce incomplete business behavior — even from AI.

# REVIEWING GENERATED BUSINESS LOGIC

*The draft moved money — the review made it correct, atomic and repeat-safe.*

## ✗ Copilot draft

```
public void transferFunds(String from, String to,
                          BigDecimal amount) {
    Account a = accounts.findById(from).get();
    Account b = accounts.findById(to).get();
    a.setBalance(a.getBalance().subtract(amount));
    b.setBalance(b.getBalance().add(amount));
    accounts.save(a);
    accounts.save(b);
    log.info("Transferred {} from {} to {}", amount, from, to);
}
```

### Missing

Transaction, amount and balance checks, account status.

## ✓ After review

```
@Transactional
public TransferResult transferFunds(TransferCommand cmd) {
    return results.findByKey(cmd.idempotencyKey()).orElseGet(() -> {
        if (cmd.amount().signum() <= 0) throw new InvalidAmountException();
        Account from = accounts.findById(cmd.from())
            .orElseThrow(() -> new AccountNotFoundException(cmd.from()));
        Account to = accounts.findById(cmd.to())
            .orElseThrow(() -> new AccountNotFoundException(cmd.to()));
        from.debit(cmd.amount()); // throws if insufficient or frozen
        to.credit(cmd.amount());
        log.info("Transfer done transferId={}", cmd.transferId());
        return results.save(TransferResult.of(cmd));
    });
}
```

### Missing

Idempotency — a retry would move money twice.

### Leak

Account numbers logged in plain text.

**KEY TAKEAWAY** Money-moving code needs a transaction, every rule, idempotency and safe logs — check each one by hand.

# EXERCISE: REVIEW AI-GENERATED TRANSFER CODE

*Decide what must be checked before this code could merge.*

## PRACTICE EXERCISE

12 MIN

### SCENARIO

Copilot generated a `transferFunds` method for Transaction Service (the draft on the previous slide). The PR author says “tests pass”.

### YOUR TASKS

- 1 Which business rules must be checked manually?
- 2 Which tests should be added?
- 3 What security issues should be reviewed?
- 4 Should the method be transactional — and what would be unacceptable?

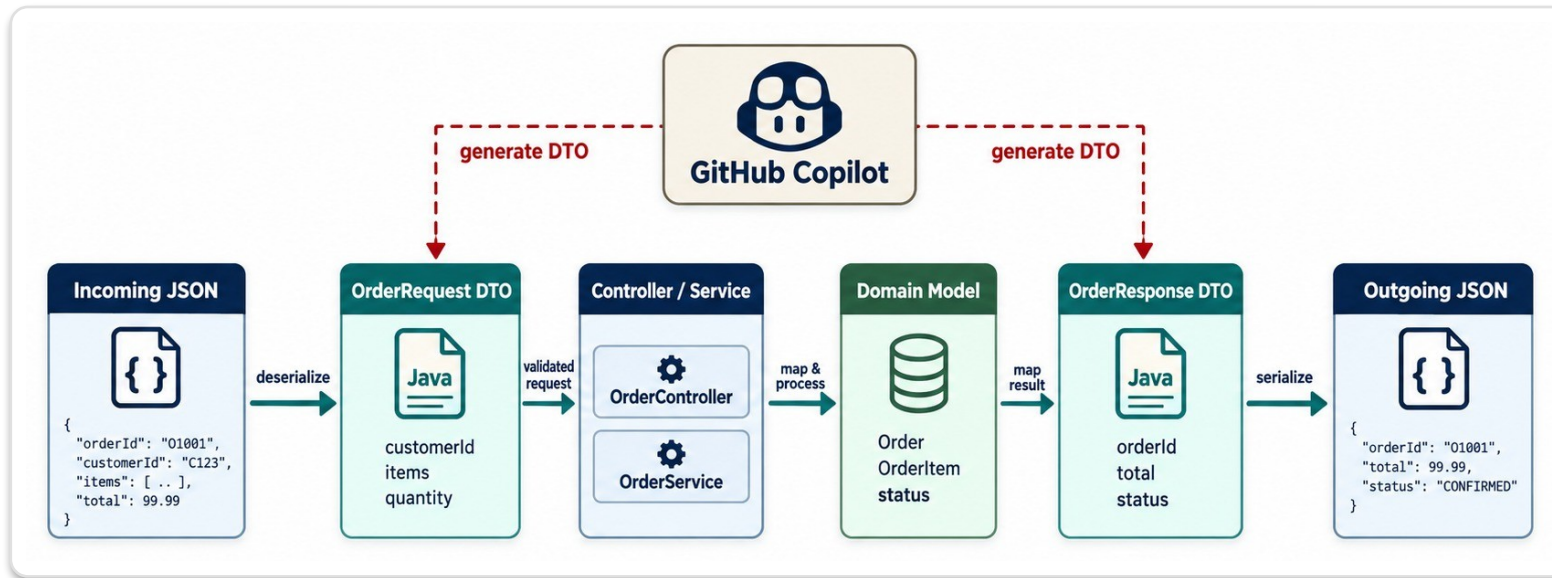
### EXPECTED OUTCOME

- ✓ Positive amount, balance, account status, limits
- ✓ Tests: insufficient funds, frozen, duplicate key
- ✓ Authorization: may this caller move this money?
- ✓ Yes, @Transactional — both updates or neither
- ✓ Unacceptable: logged account numbers, `get()`

**KEY TAKEAWAY** “Tests pass” proves little if the tests and the rules came from the same prompt.

# GENERATING DTOS AND API CONTRACTS

*Request/response DTOs, validation annotations and stable API boundaries.*



## ✓ Good boilerplate for Copilot

```
public record CreateOrderRequest(
    @NotBlank String customerId,
    @NotEmpty List<@Valid Item> items) {}
```

## Check

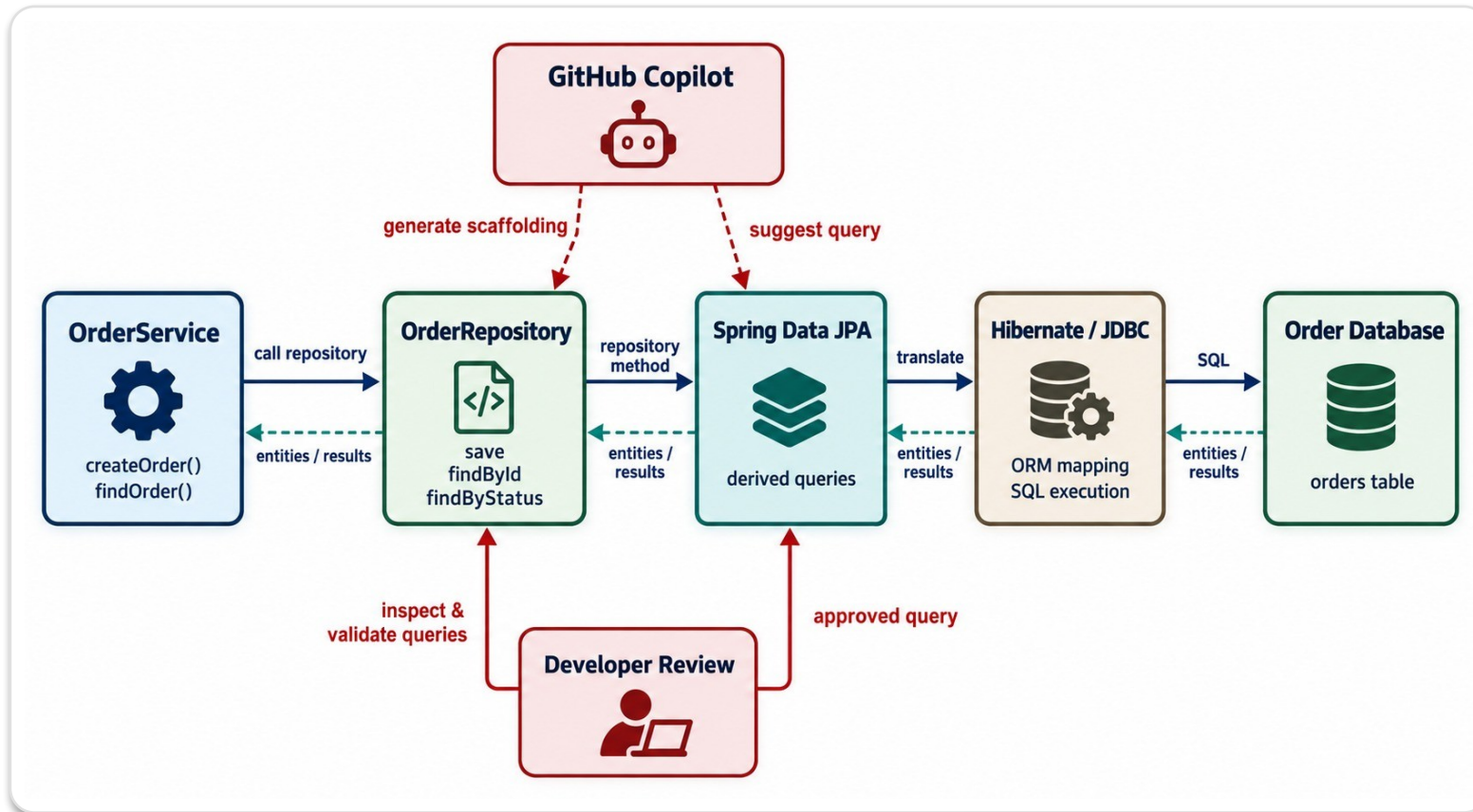
Only the fields clients need — no IDs or audit fields in requests.

**MODULE 3** DTOs keep the contract separate from entities.

**KEY TAKEAWAY** Let Copilot write DTO boilerplate — you keep the API contract separate from persistence.

# GENERATING REPOSITORY AND PERSISTENCE CODE

*Spring Data JPA repositories, query methods and persistence concerns.*



## COMMON GENERATED PROBLEMS

- ! Fields that don't exist
- ! N+1 queries
- ! `findAll()` on big tables
- ! Missing indexes

### How to check

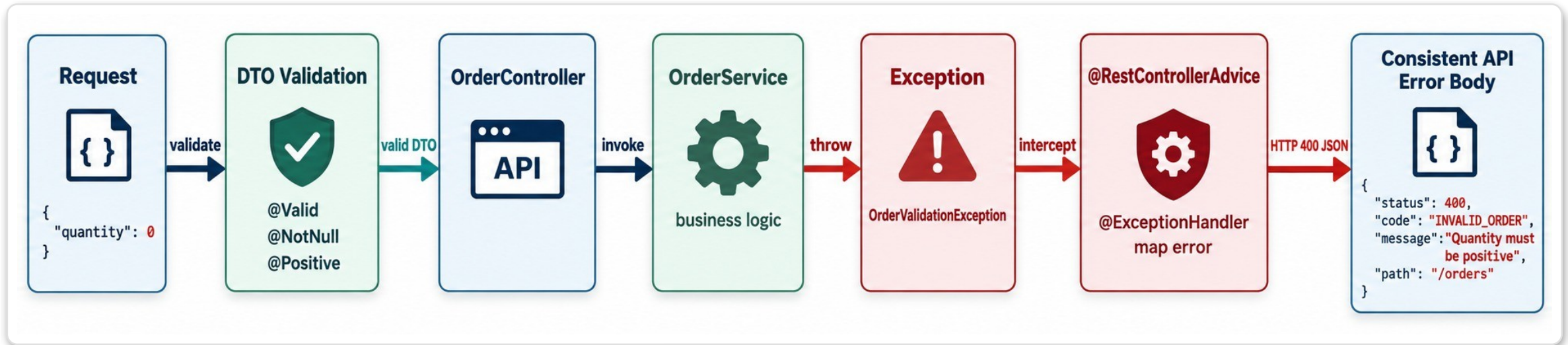
Turn on SQL logging in a test and read the queries it runs.

**MODULE 5** Relationships stop at the service boundary.

**KEY TAKEAWAY** Generated persistence code compiles easily — review its SQL, indexes, volume and N+1 risk.

# USING COPILOT FOR ERROR HANDLING AND VALIDATION

*@Valid, exception handling and API error models.*



## Ask for one strategy

One `GlobalExceptionHandler` and one error record for every endpoint.

## REJECT

- ✗ Stack traces in responses
- ✗ 500 for validation errors
- ✗ SQL details in messages

**MAP** 400 validation · 404 missing · 409 conflict.

**KEY TAKEAWAY** Ask for one consistent error strategy — and review it for leaks and wrong status codes.

# PART 3: AI-ASSISTED QUALITY

*Use Copilot to test, refactor, review and debug — with proof.*



## SO FAR

- ✓ Generate and review one layer at a time
- ✓ Business logic checked rule by rule
- ✓ DTOs, queries and errors reviewed

## NEXT QUESTION

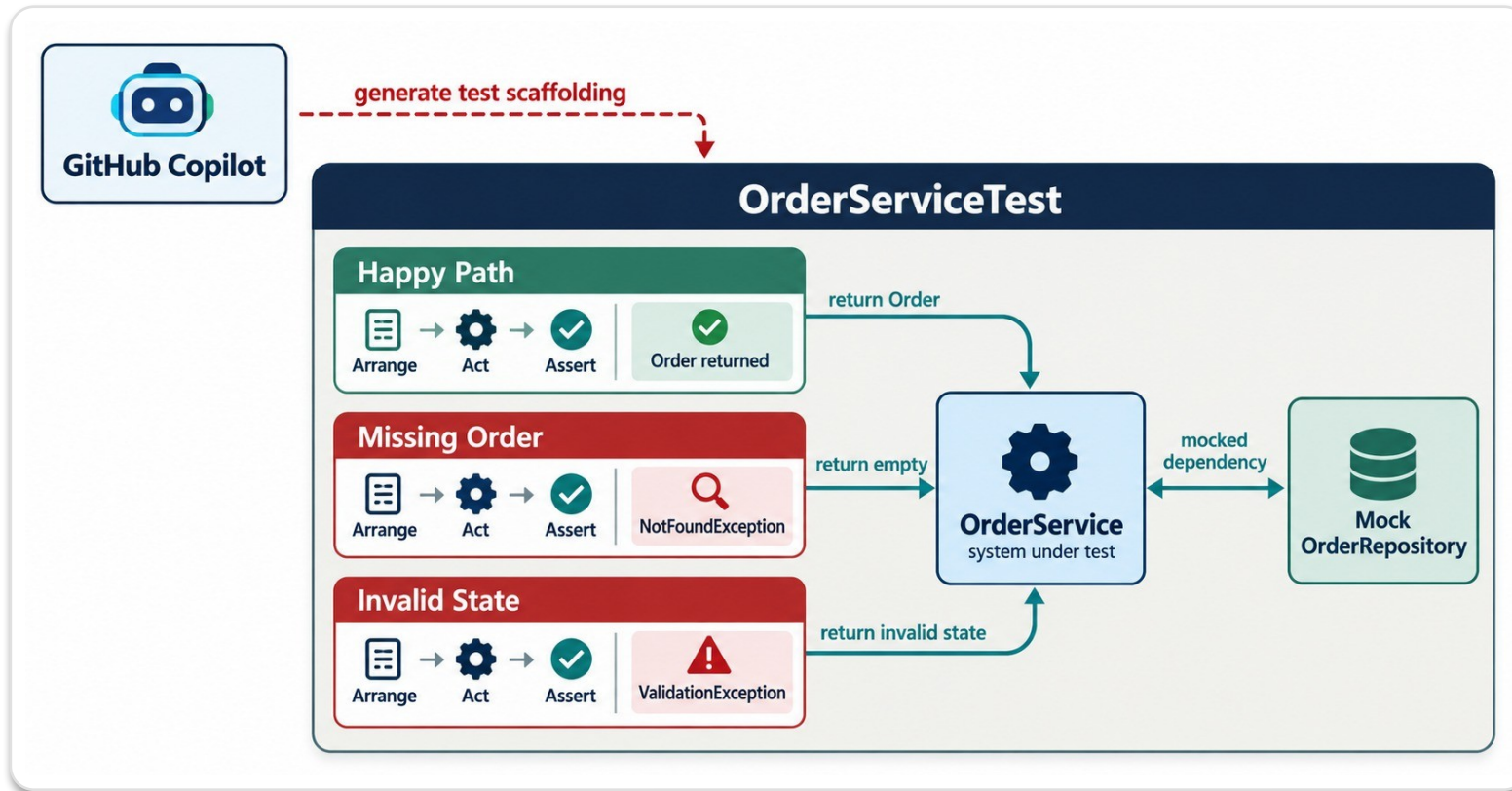
**Can AI help prove the code is right — and keep it clean?**

## IN THIS PART

- 1 Unit and controller tests — with an example
- 2 Finding test gaps
- 3 Testing a policy table — and Exercise 5.2
- 4 Refactoring and code review
- 5 Troubleshooting — with an exercise

# GENERATING UNIT TESTS WITH COPILOT

*JUnit, Mockito and service-layer tests.*



## Prompt for cases

"Test happy path, missing order and invalid state for cancelOrder."

## Arrange-Act-Assert

Keep each test to one behavior.

**ASK** Would this test fail if the code were wrong?

**KEY TAKEAWAY** Let Copilot scaffold tests, then check that they actually verify the behavior.

# A TEST THAT PROVES NOTHING — AND ONE THAT DOES

*Generated tests often assert that code runs, not that it's correct.*

## ✗ Generated

```
@Test
void cancelOrderWorks() {
    when(repo.findById(1L))
        .thenReturn(Optional.of(order));

    service.cancelOrder(1L);

    assertNotNull(order);
}
```

### Weak

assertNotNull passes even if cancel does nothing.

## ✓ After review

```
@Test
void paidOrderIsCancelledAndRefundRequested() {
    Order order = Order.withStatus(PAID);
    when(repo.findById(1L)).thenReturn(Optional.of(order));

    service.cancelOrder(1L);

    assertThat(order.getStatus()).isEqualTo(CANCELLED);
    verify(events).publish(any(OrderCancelled.class));
}

@Test
void shippedOrderCannotBeCancelled() {
    when(repo.findById(1L))
        .thenReturn(Optional.of(Order.withStatus(SHIPPED)));
    assertThatThrownBy(() -> service.cancelOrder(1L))
        .isInstanceOf(InvalidOrderStateException.class);
}
```

### Strong

Asserts the new state and the published event.

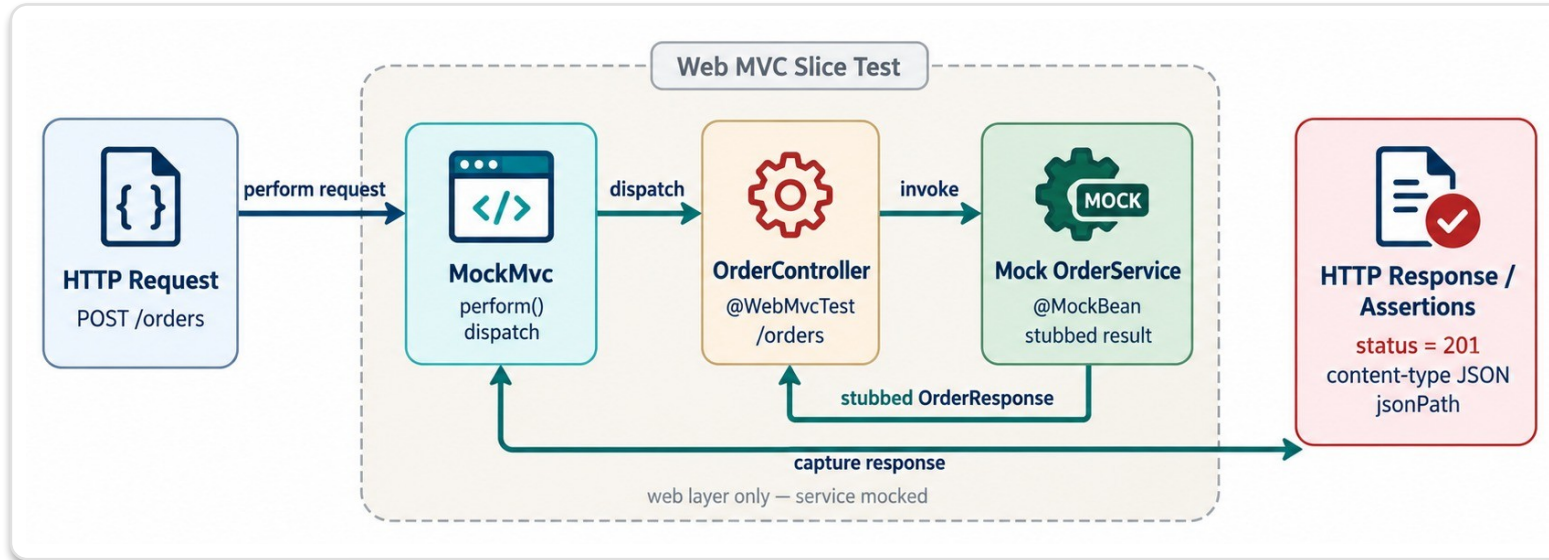
### Negative case

Proves the rule that shipped orders can't cancel.

**KEY TAKEAWAY** A test is only useful if it fails when the behavior is wrong — check every generated assertion.

# GENERATING CONTROLLER AND API TESTS

*MockMvc/WebMvcTest, HTTP status and JSON assertions.*



## An HTTP contract test

```
mvc.perform(post("/api/v1/orders")
    .contentType(APPLICATION_JSON)
    .content("{}"))
    .andExpect(status().isBadRequest());
```

## Covers

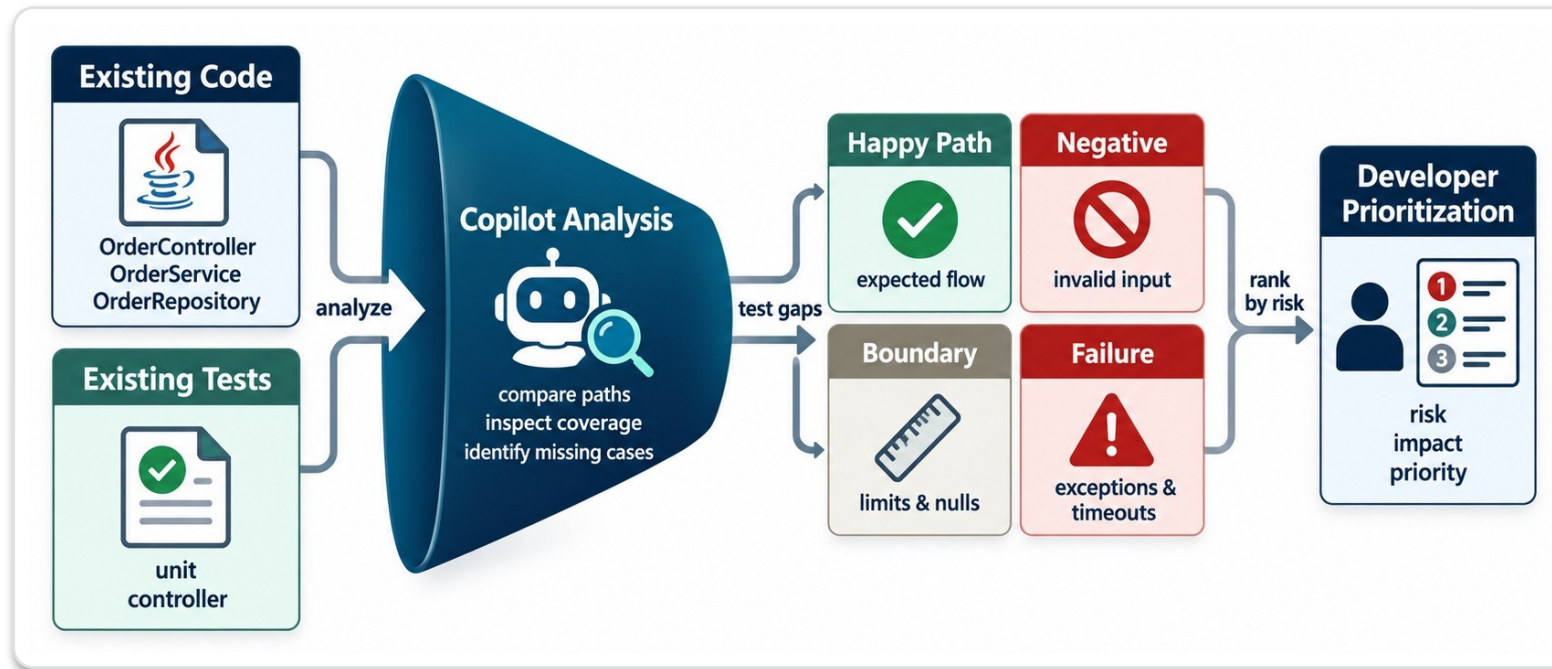
Status codes, JSON shape, validation and security rules.

**TWO QUESTIONS** Service: is the logic right?  
Controller: is the contract?

**KEY TAKEAWAY** Service tests check business logic; controller tests check the HTTP contract.

# IMPROVING TEST COVERAGE WITH COPILOT

*Edge cases, negative tests and boundary conditions.*



## Prompt

"List untested behaviors in OrderService, grouped by risk."

## GAPS IT OFTEN FINDS

Negative

Boundary

Failure

Concurrency

**COVERAGE  $\neq$  QUALITY** Prioritize risk, not a percentage.

**KEY TAKEAWAY** Use Copilot to find the missing high-value tests — not to chase a coverage number.

# PROVING A POLICY TABLE WITH TESTS

*Lab 5's policy-v1 rules as deterministic Java — and one test row per rule.*

## PolicyEngine.java

```
public Decision decide(ModelResult model, BigDecimal amount) {
    if (!model.ok())
        return Decision.hold("MODEL_UNAVAILABLE");
    if (amount.compareTo(policy.highValue()) >= 0)
        return Decision.hold("HIGH_VALUE");
    if (model.score() >= policy.declineScore())
        return Decision.decline("HIGH_SCORE");
    if (model.score() < policy.approveScore()
        && amount.compareTo(policy.lowValue()) < 0)
        return Decision.approve("LOW_SCORE_LOW_VALUE");
    return Decision.hold("REVIEW_BAND");
}
```

## ✓ PolicyEngineTest.java

```
@ParameterizedTest
@CsvSource({
    "false, 10, 100, HOLD, MODEL_UNAVAILABLE",
    "true, 10, 5000, HOLD, HIGH_VALUE",
    "true, 70, 100, DECLINE, HIGH_SCORE",
    "true, 39, 999, APPROVE, LOW_SCORE_LOW_VALUE",
    "true, 40, 999, HOLD, REVIEW_BAND"
})
void appliesPolicyV1(boolean ok, int score, BigDecimal amount,
    Disposition expected, String reason) {
    Decision d = engine.decide(new ModelResult(ok, score), amount);
    assertThat(d.disposition()).isEqualTo(expected);
    assertThat(d.reason()).isEqualTo(reason);
}
```

### First match wins

Order of the if statements is the policy.

### Boundaries

39 vs. 40 and 4999 vs. 5000 are where bugs hide.

### Thresholds

From configuration — no magic numbers.

**KEY TAKEAWAY** Let Copilot draft the rules, then let a test row per rule — including model failure — prove them.

# EXERCISE 5.2 — APPROVE / HOLD / DECLINE POLICY RULES

Day 5 checkpoint B: synthetic approve, hold and decline thresholds.

OFFICIAL CHECKPOINT B

15 MIN

labs/day-05/exercises/exercise-5.2-policy-rules.md

## SCENARIO

Lab 5's Risk Assessment Service gets a score from a model, then a deterministic Java PolicyEngine decides the disposition.

Scope: Account, Transaction and Risk Assessment only.

## YOUR TASKS

- 1 First match wins: model not OK → HOLD MODEL\_UNAVAILABLE
- 2 Amount  $\geq 5000$  → HOLD HIGH\_VALUE; score  $\geq 70$  → DECLINE HIGH\_SCORE
- 3 Score  $< 40$  and amount  $< 1000$  → APPROVE LOW\_SCORE\_LOW\_VALUE; else HOLD REVIEW\_BAND
- 4 Use GitHub Copilot (Free, on the VM) to draft the if/else — review before accepting; tests must prove the table

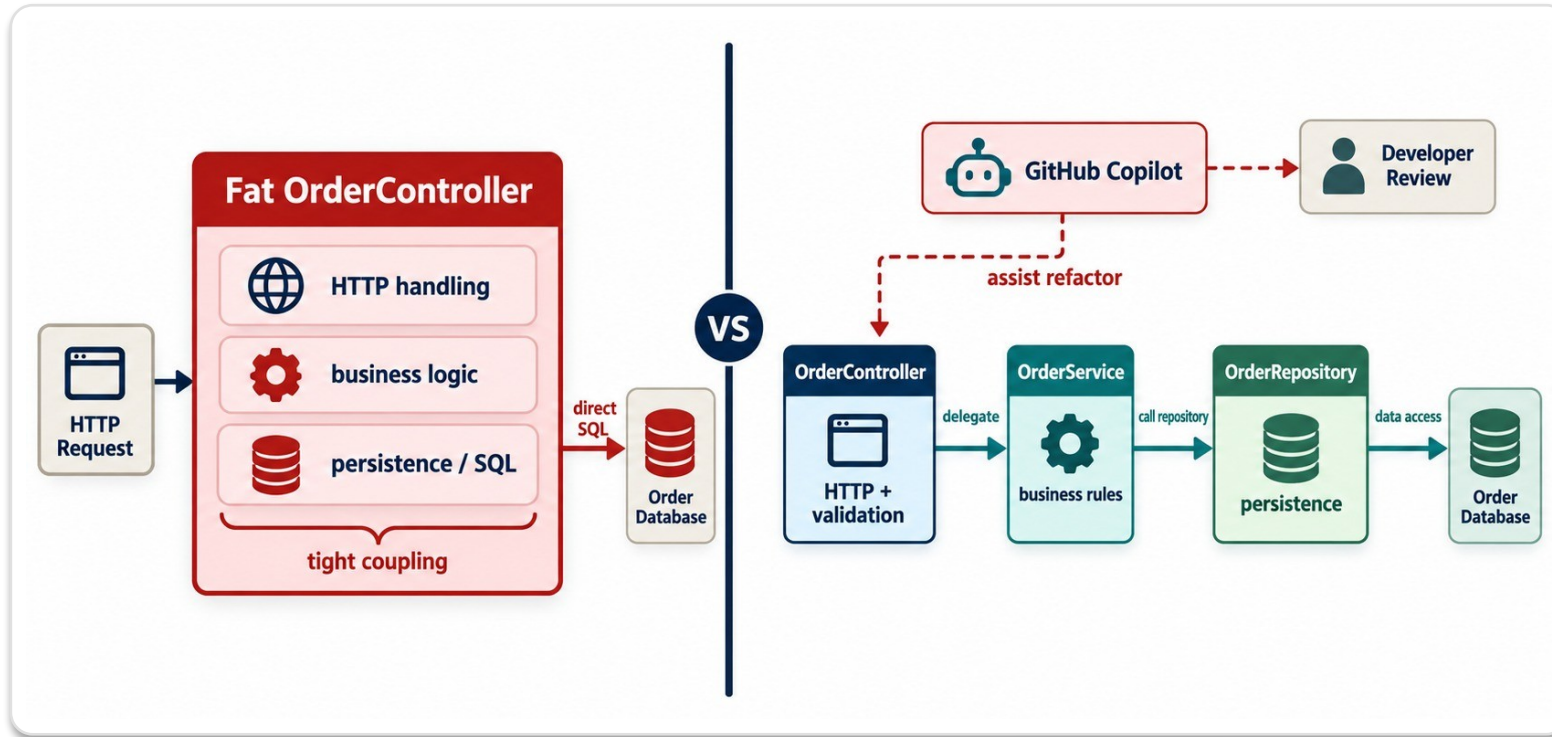
## EXPECTED OUTCOME

- ✓ The policy-v1 table in Java
- ✓ One test per row
- ✓ Model failure tested
- ✓ Implemented in Lab 5 Step 3

**KEY TAKEAWAY** The policy-v1 table written in Java, with a test for every row — including model failure.

# REFACTORING CODE WITH COPILOT

*Simplification, duplication removal, method extraction and naming.*



## Analyze first

"What responsibilities does this class have?"

## One change at a time

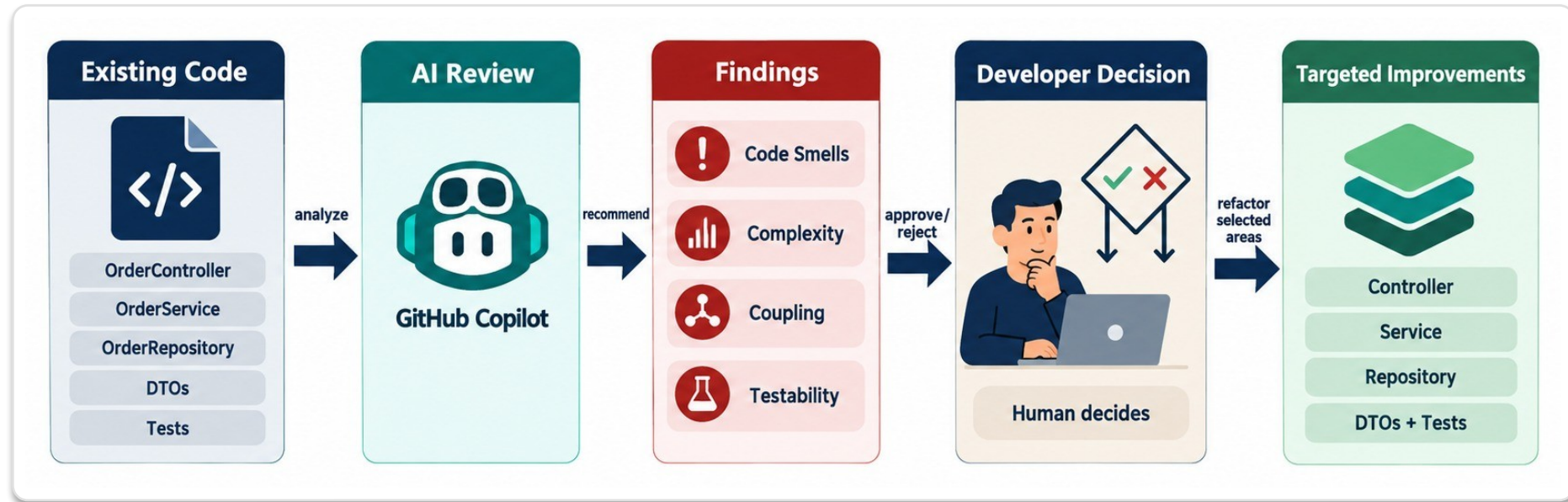
Extract the service, run tests; move queries, run tests again.

**SAFETY NET** No tests yet? Write them before refactoring.

**KEY TAKEAWAY** Refactor incrementally with Copilot — analyze, propose, review, then change one thing.

# USING COPILOT FOR CODE REVIEW AND CODE QUALITY

*Smells, complexity, maintainability and improvement suggestions.*



## USEFUL FINDINGS

Long methods

Duplication

Tight coupling

Null handling

## Challenge it

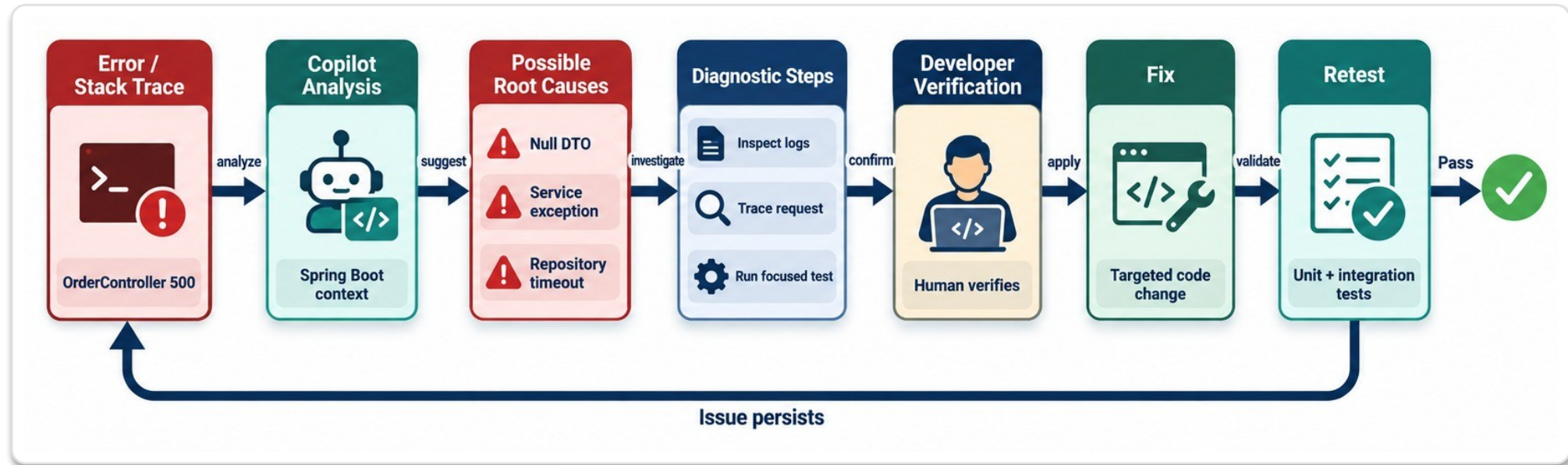
Ask why — and reject changes that alter behavior.

**WHO DECIDES** The human reviewer still approves the PR.

**KEY TAKEAWAY** Use Copilot as an extra reviewer — the developer still decides what changes.

# USING COPILOT FOR TROUBLESHOOTING

*Explain stack traces and diagnose Spring Boot startup and configuration issues.*



## Give it evidence

The full stack trace, the config and what changed — not “fix this”.

## COMMON STARTUP PROBLEMS

Missing bean

Bad property

Port in use

DB refused

**HABIT** Verify the root cause before applying a fix.

**KEY TAKEAWAY** Diagnose first: separate root cause from noise, verify, then make the smallest fix.

# EXERCISE: TROUBLESHOOT A STARTUP FAILURE

*Use Copilot as a diagnostic partner — then verify its answer.*

## PRACTICE EXERCISE

10 MIN

### SCENARIO

Order Service fails at startup after a teammate's merge:

```
APPLICATION FAILED TO START
Parameter 0 of constructor in
    com.example.order.web.OrderController
required a bean of type
    com.example.billing.PaymentClient
that could not be found.
```

### YOUR TASKS

- 1 Write the prompt you'd give Copilot
- 2 List the likely causes it should consider
- 3 Say how you'd verify before changing code

### EXPECTED OUTCOME

- ✓ Prompt: error, package layout, recent change
- ✓ Package outside component scan
- ✓ Missing @Component or @Bean
- ✓ Profile-conditional bean not active
- ✓ Verify with --debug and the bean list

**KEY TAKEAWAY** Give Copilot the evidence, weigh its hypotheses, and verify the cause before you fix anything.

# PART 4: ENTERPRISE GOVERNANCE

*Speed is only valuable if safety, quality and accountability stay intact.*



## SO FAR

- ✓ Tests must fail when behavior is wrong
- ✓ Refactor in small, tested steps
- ✓ Troubleshooting starts with evidence

## NEXT QUESTION

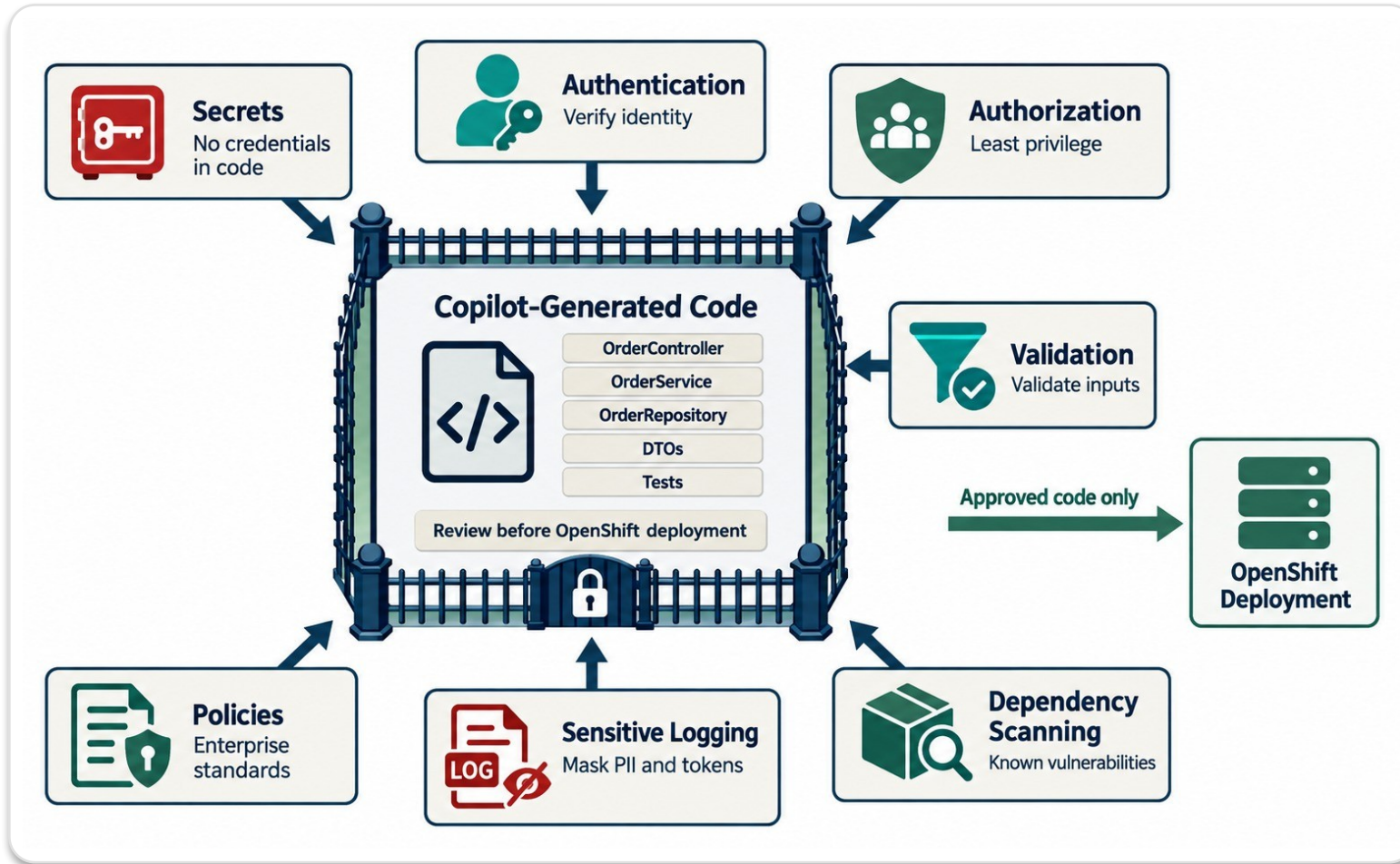
**How does an enterprise team use Copilot safely — and who is accountable?**

## IN THIS PART

- 1 Security guardrails
- 2 Enterprise best practices
- 3 What Copilot should not decide
- 4 The hands-on lab workflow
- 5 The complete AI-assisted architecture

# SECURITY AND ENTERPRISE GUARDRAILS

*Secrets, insecure code, auth, dependency risk, sensitive data and validation.*



## NEVER PUT IN PROMPTS

- ✗ Customer or account data
- ✗ Passwords, tokens, keys
- ✗ Internal URLs and confidential code

## Invented dependencies

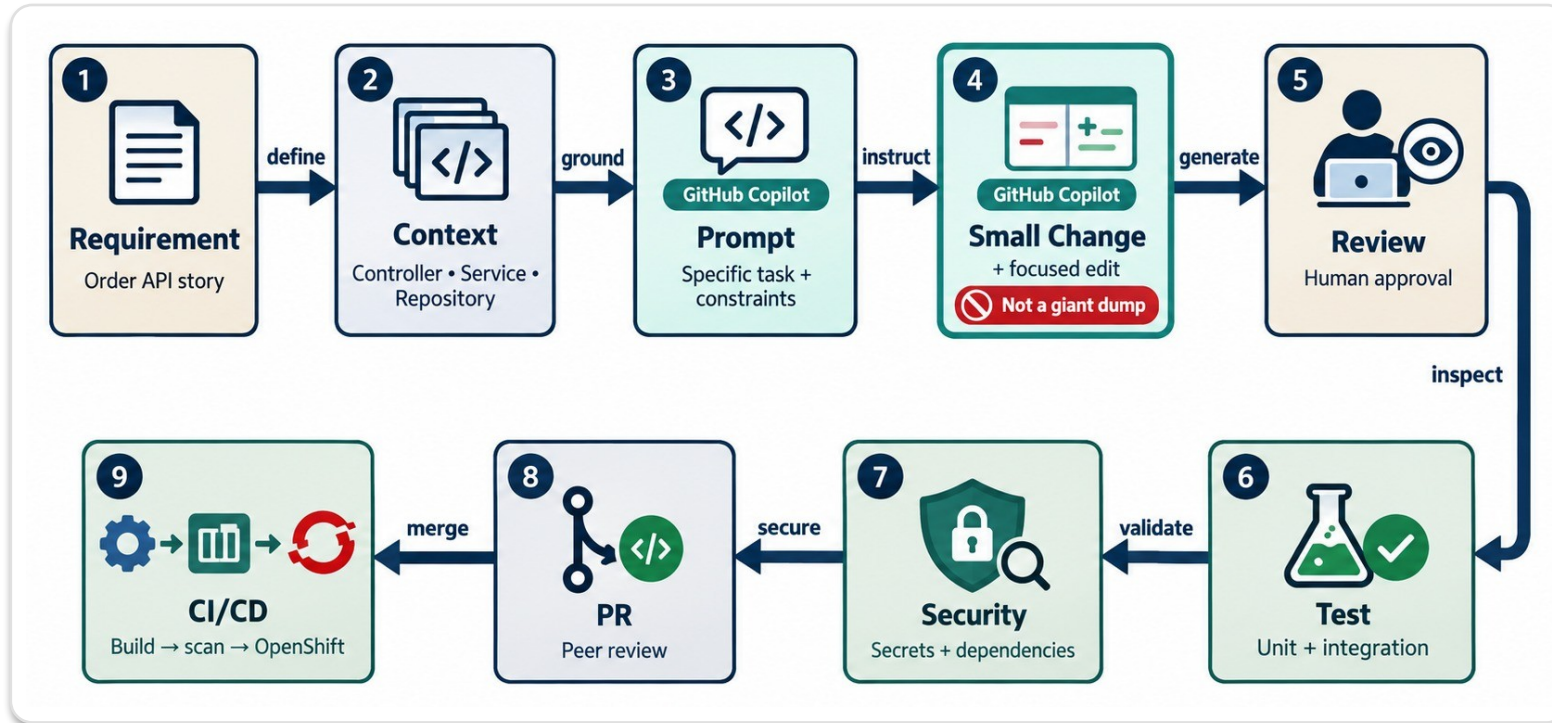
Check that suggested libraries and APIs actually exist.

**SAME GATES** Scans, review and tests apply to AI code.

**KEY TAKEAWAY** AI-generated code passes the same security gates as any other code.

# COPILOT BEST PRACTICES FOR ENTERPRISE DEVELOPMENT

*Use Copilot to accelerate — not bypass — the engineering process.*



## TEAM HABITS

- ✓ Small, reviewable changes
- ✓ Understand before accepting
- ✓ Tests for every generated behavior
- ✓ PR and CI gates unchanged

## Measure the right thing

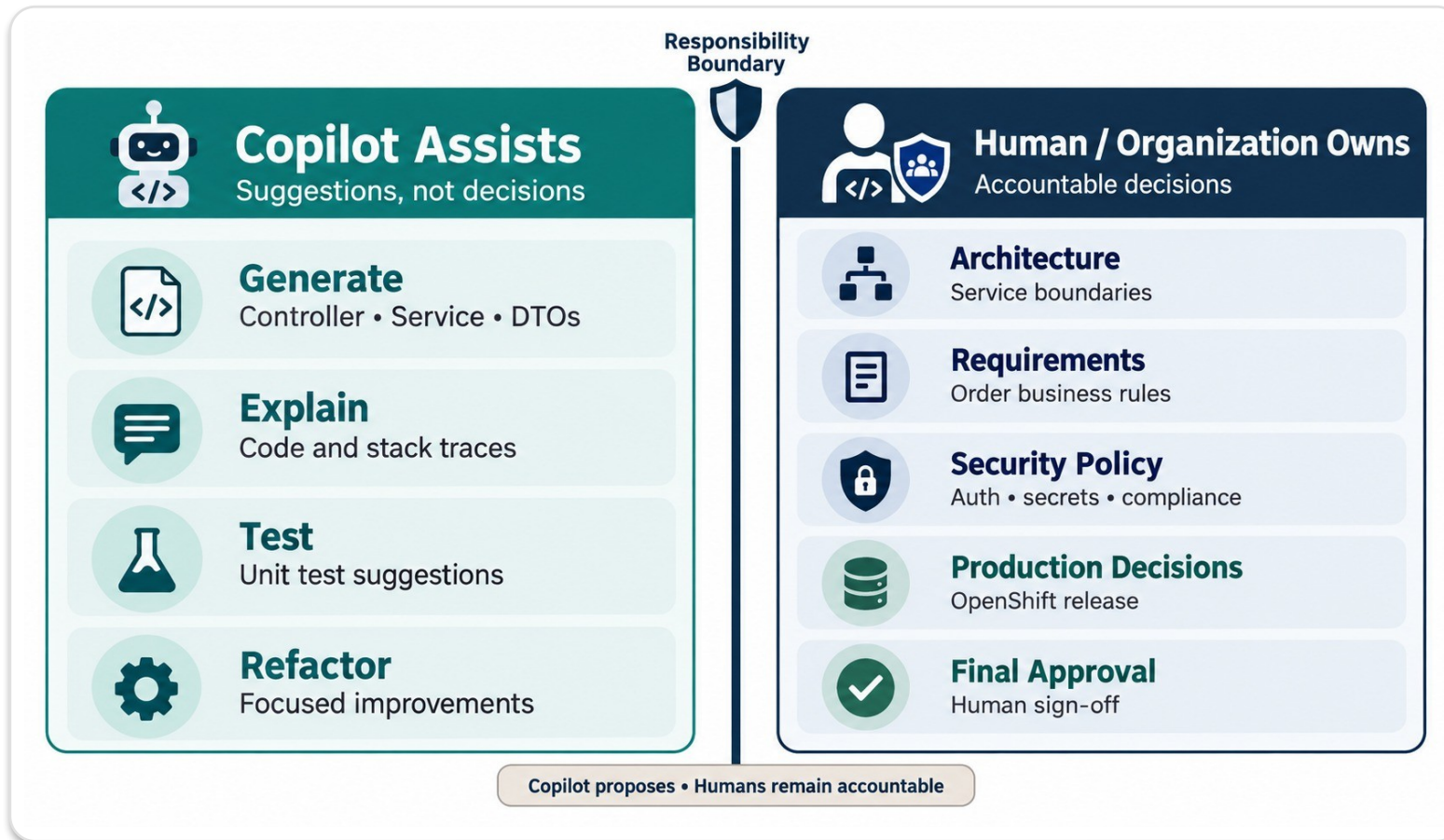
Lead time and defect rates — not lines of code generated.

**GOVERNANCE** Org policies control features and data use.

**KEY TAKEAWAY** Use AI for speed while keeping every existing engineering control.

# WHAT COPILOT SHOULD NOT DECIDE FOR YOU

*Architecture, business correctness, security policy and production approval.*



## Modules 2 and 4

Service boundaries and integration style.

## Module 5

Data ownership and transaction design.

## Modules 6, 7, 8 and 12

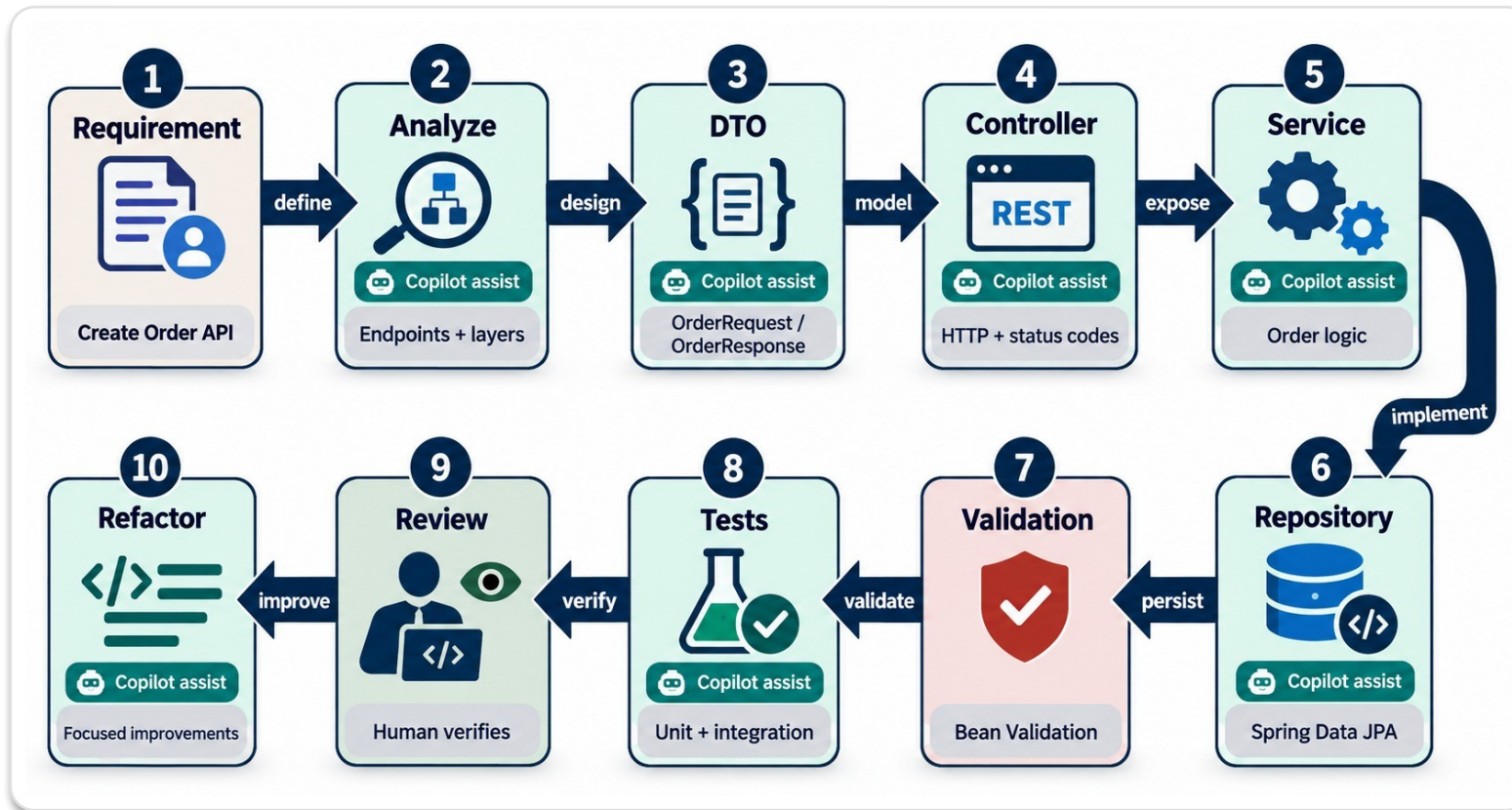
Resilience policy, release approval and security.

**MESSAGE** Accountability stays with people.

**KEY TAKEAWAY** AI doesn't replace the architectural principles learned in Modules 1–8.

# HANDS-ON LAB — BUILD AN ORDER MICROSERVICE WITH COPILOT

*A guided, incremental AI-assisted development workflow.*



## DEFINITION OF DONE

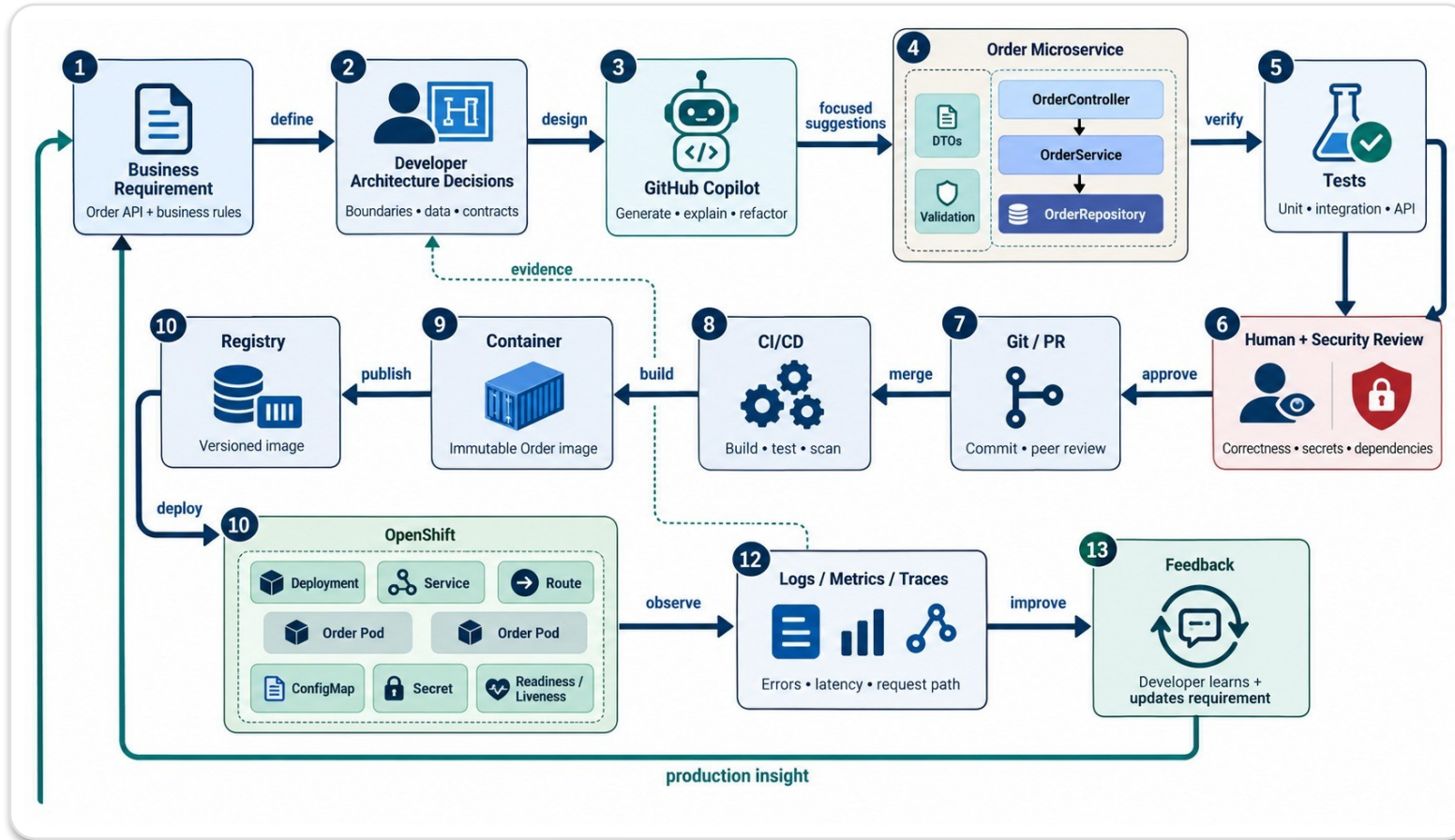
- ✓ Endpoints match the contract
- ✓ Validation and errors tested
- ✓ No entity leaks or secrets
- ✓ Tests fail when code breaks
- ✓ Reviewed like any PR

**DON'T** use one giant prompt for the whole service.

**KEY TAKEAWAY** Work incrementally: plan, prompt, generate, review, test, refine — one layer at a time.

# END-TO-END AI-ASSISTED DEVELOPMENT WORKFLOW

*From requirement to production feedback, with Copilot inside every step.*



## EVERY PART OF THE COURSE

- ✓ People own requirements and architecture
- ✓ Copilot drafts code and tests
- ✓ Review, security and CI/CD gate it
- ✓ OpenShift runs it; observability feeds back

**REMEMBER** Accelerator, not authority.

**KEY TAKEAWAY** Use GitHub Copilot as an engineering accelerator, not an engineering authority.

# COPILOT MISCONCEPTIONS AND ANTI-PATTERNS

*Beliefs and habits that turn AI speed into production defects.*

## MISCONCEPTION

## REALITY

## ANTI-PATTERNS TO AVOID

**It compiles, so it's correct**

Compiling code can break every business rule

**Generated tests prove the code works**

Weak assertions prove nothing

**A good prompt removes the need to review**

Better candidates still need review

**Copilot knows our architecture**

Only what you give it as context

**AI is responsible for AI-written bugs**

The team owns every merged line

- ✗ One giant prompt for a whole service
- ✗ Pasting real customer data into prompts
- ✗ Accepting unknown dependencies
- ✗ Skipping review for "simple" code
- ✗ Chasing coverage percentages
- ✗ Letting AI choose service boundaries

**REMEMBER** Accelerator, not authority.

**KEY TAKEAWAY** AI speeds up good engineering habits — and multiplies the damage of bad ones.

# KNOWLEDGE CHECK

*Answer in your own words — then compare with a partner.*

1 What is GitHub Copilot used for?

2 Why is prompt clarity important?

3 What should a controller prompt include?

4 Why review generated service code carefully?

5 One risk of generated repository methods?

6 Why aren't generated tests automatically trustworthy?

7 What is refactoring?

8 What must never go into prompts?

9 What does AI hallucination mean?

10 Who is responsible for final code quality?

# MODULE 9 SUMMARY AND WHAT'S NEXT

*From vague prompts to reviewed, tested, AI-assisted Spring Boot code.*

## YOU CAN NOW

- ✓ Structure prompts with context, rules, errors and output
- ✓ Generate Spring Boot layers one at a time and review them
- ✓ Turn weak generated tests into ones that prove behavior
- ✓ Refactor, review and troubleshoot with Copilot
- ✓ Apply enterprise guardrails and keep accountability

## NEXT: MODULE 10 — AGENTIC AI WITH MICROSOFT SERVICES

- 1 What makes an AI system an agent?
- 2 How do agents call tools and business APIs safely?
- 3 Where do humans stay in the loop?

**BRING ALONG** Your Lab 5 PolicyEngine — agents must respect rules like it.

**KEY TAKEAWAY** Module 10 moves from AI that writes code to AI that takes actions — under control.